

Data Engineering and House Price Prediction with Pyspark

In this project, we integrate economic data at the district and neighborhood level with property listings from Idealista to build a predictive model for housing prices. The Idealista dataset provides detailed information per listing, including price, number of rooms, size, and address-level location data.

To enhance these records with socioeconomic context, we incorporate neighborhood-level indicators such as average family income and unemployment counts. This enriched dataset enables us to train a predictive model to estimate housing prices based on both property and demographic attributes.

A key preprocessing step involved estimating income data for the year 2020. Since the most recent available income data was from 2017, we project income values to 2020 to match the temporal keys of the Idealista and unemployment datasets, both of which refer to 2020.

To streamline the entire data processing and modeling workflow, we implemented an automated pipeline using **Apache Airflow** as described in the end. The pipeline is composed of:

- **format_raw_data** – loads raw data from the Landing Zone, normalizes and cleans it, and exports the standardized datasets to the Formatted Zone. This is done in the file *Codes/raw_formatted.py* and corresponds to A.2 .
- **enrich_data** – joins datasets and enriches features using lookup tables and socioeconomic information, outputting the result to the Exploitation Zone. This is done in the file *Codes/formatted_exploitation.py* and corresponds to A.3 .
- **train_and_deploy_model** – performs feature engineering, trains two predictive models, and logs them using MLflow. This is done in the file *Codes/Analysis.py*

The orchestration DAG (**data_pipeline_dag**) is defined using **BashOperator** tasks in Airflow and ensures that each job is executed in the correct order. The figure below shows the logical task flow:

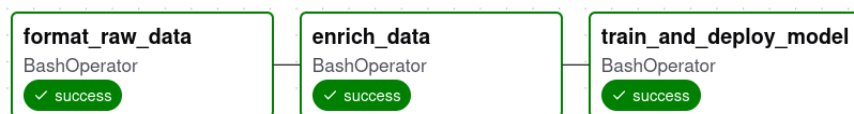


Figure 1: Airflow DAG execution graph

A: Data Management Backbone

A.1: Explore the data and choose the KPI's

We explored the following datasets: **1. Idealista Housing Prices .json**, **2. Open Data BCN Unemployment Dataset .json**, **3. Income and Unemployment Lookup Dataset** (used for joining and getting IDs of neighbourhoods) .csv

After examining the available features, we selected the following key performance indicators (KPIs) for modeling the housing prices in Barcelona. We will basically use all of the features available in the Idealista Dataset along with the unemployment and income from the other datasets. These include the property price per square meter ($\text{€}/\text{m}^2$), which serves as the target variable, average family income as a socioeconomic indicator by neighborhood, and annual unemployment figures representing the total unemployed population per neighborhood. We also incorporate various property features such as the number of rooms, number of bathrooms, and size in square meters (m^2).

To combine information from various dataframes, we used a set of keys including standardized numeric identifiers such as **district_id** and **neighborhood_id**, as well as normalized district and neighborhood names to reconcile naming mismatches across datasets.

A.2: Data Formatting Process

We implemented the following using **PySpark** to handle raw data ingestion, schema normalization, and export to a unified format into the Formatted Zone. We apply the following steps in this zone. The code can be seen in **raw_formatted.py**

Step 1: Load Raw Data – Datasets from the Landing Zone were loaded using Spark reader. This included: Income data **year_Distribucio_territorial_renda_familiar.csv**, Unemployment data **year_atur_per_sexe.json**, Idealista JSON property listings (2020) **year_month_day_idealista.json** and Lookup tables mapping neighborhoods and districts

Step 2: Standardize Columns and Types – All datasets were cleaned and normalized by renaming key columns (e.g., `Nom_Barri` to `neighborhood`), casting numerical fields to appropriate Spark types such as `int` and `double`, and dropping obsolete columns like `_id`, which were not being used.

Step 3: Normalize Geographic Names – To reconcile inconsistent naming, we applied normalization UDFs to strip accents, unify case, and remove extra spacing. To verify the effectiveness of this process, we matched districts and neighborhoods against the lookup tables using inner joins.

Step 4: Export to Formatted Zone – The cleaned and normalized DataFrames were exported to the Formatted Zone in Parquet format using `.write.mode("overwrite").parquet(...)`.

This included `income.parquet`, `unemployment.parquet`, `idealista.parquet`, `lookup_idealista.parquet`, `lookup_income.parquet`.

A.3 Move data to Exploitation Zone

Once the data was fully standardized, we load the data from the formatted zone and go through the following steps in the script `formatted_exploitation.py`.

Step 1: Enrich with definite geographic (neighborhood/district) identifiers – We joined each dataset with the corresponding lookup table on normalized neighborhood and district names to attach the standardized `district_id` and `neighborhood_id`.

Step 2: Aggregate Unemployment – We grouped the unemployment data by `district_id` and `neighborhood_id` and computed the total number of unemployed individuals per neighborhood.

Step 3: Join the Datasets – The unemployment indicators were merged into the Idealista dataset using joins on geographic IDs. This resulted in a fully enriched feature matrix with Listing-level attributes (rooms, size, price, etc.) and Neighborhood-level indicators (unemployment, population).

Note: The Income is not yet in this final aggregated dataset because we need it in long format to model the income in 2020.

Step 5: Export to Exploitation Zone – The final merged datasets were written to the Exploitation Zone as `idealista_unemp.parquet` and `income_extended.parquet`. This file serves as the basis for all predictive modeling tasks in the next stage. We can do analysis on this code now, meaning we create `income_2020_extended` and can add this to the final aggregated dataframe then. However, this will be done in the `analysis.py` file as there should be no analysis/modelling at the current stage.

A.4: Validate the Data

To validate the data integrity across transformations, we performed multiple checks both during and after the formatting process.

Join Validation During Normalization – While standardizing district and neighborhood names, we performed inner joins with the corresponding lookup tables to ensure that each row could be mapped to a valid geographic identifier. This allowed us to detect mismatches in naming conventions early on.

Record Counts and Filtering Checks – For each dataset, we compared row counts before and after filtering nulls, and before and after joins with lookup tables. This helped us confirm that the majority of records were retained and properly matched.

Column Type Consistency – We explicitly cast numerical columns to the correct types (e.g., `double`, `int`) and validated schema consistency using `.printSchema()` and `.summary()` methods in Spark.

Sample-Based Sanity Checks – We manually inspected small samples of joined datasets using `.show()` and filtered queries (e.g., neighborhoods starting with specific strings).

These validations ensured the reliability of the final matrix exported to the Exploitation Zone for predictive modeling.

B: Predictive Analysis via Model Training and Management

In this section, we develop and evaluate predictive models to estimate housing prices (€/m²) using property features and socioeconomic indicators.

B.1: Income Projection for 2020

To align our datasets temporally, we extended the income data—originally available only up to 2017—to the year 2020. Since both the Idealista listings and unemployment data refer to 2020, we estimated the missing income values for 2018, 2019, and 2020 using a rolling average approach in PySpark. Specifically, for each `neighborhood_id`, we applied a three-year rolling average ordered by year to smooth out income trends. The average of the three most recent years was used to forecast the next year's income, repeating this process iteratively for 2018 through 2020.

This approach preserved neighborhood-specific income dynamics while mitigating short-term noise or missing data. We validated the forecasts through line plots, confirming that projected values followed prior trends smoothly. The estimated income for 2020 was then merged with the Idealista–Unemployment dataset to support downstream modeling.

For neighborhoods lacking sufficient historical data, we imputed missing forecasts using the district-level average income to ensure complete coverage. The final enriched dataset includes property-level features (e.g., rooms, size, price, property type, location) and socioeconomic indicators, with **priceByArea** (€/m²) as the modeling target.

Feature Engineering:

We selected relevant features from the original dataset. This involved extracting subfields from nested columns such as **parkingSpace** and **detailedType**, and keeping a broad range of property and contextual features. We also retained boolean flags such as **hasLift**, **hasPlan**, and **newDevelopment**.

Tree-based models are inherently **scale-invariant**; they split data using feature thresholds rather than relying on the absolute scale of features. Therefore, no normalization was performed.

Missing Value Handling:

Before training, we cleaned the dataset and addressed missing values using the following logic: **subTypology** was dropped due to excessive missingness; **hasParkingSpace** and **parkingIncluded** were imputed as **False** when null; **parkingPrice** was set to 0 when both **hasParkingSpace** and **parkingIncluded** were true or when its value was null; **floor** values were imputed as **"unknown"**; and **hasLift** was imputed as **False**, based on the assumption that missingness indicates absence or lack of mention of an elevator.

After preprocessing, the final modeling dataset consisted of 6,290 records and 28 features, ready for training and evaluation.

Predictive Analysis: Modeling House Prices

To model housing prices in Barcelona, we focused on predicting **priceByArea** (price per square meter) as the target variable, rather than absolute price, to normalize for property size and support more interpretable comparisons.

We selected two tree-based models for this task: the **Gradient Boosted Trees Regressor (GBTRegressor)** and the **Random Forest Regressor**. Tree-based ensemble methods are particularly well-suited for capturing complex, nonlinear relationships and feature interactions common in real estate markets.

Categorical features such as **location** and **typology** were encoded using a two-step process: **StringIndexer** followed by **OneHotEncoder**. These encoded vectors were combined with numerical and boolean features into a single feature vector using **VectorAssembler**, forming the input for model training.

Gradient Boosted Trees Regressor (GBT): We trained a **GBTRegressor** using a full PySpark ML pipeline. The pipeline included: indexing and encoding of categorical variables, feature vector assembly, Model training with gradient boosting.

To assess model performance, we randomly split the dataset into training (80%) and testing (20%) subsets using a fixed random seed for reproducibility. After training the model on the training set, we generated predictions on the test set.

Performance was evaluated using the Root Mean Squared Error (RMSE), a standard metric for regression tasks that penalizes large errors. The **GBTRegressor** was configured with **maxIter=100** boosting iterations.

Random Forest Regressor: To provide a baseline for comparison, we also trained a **RandomForestRegressor** using the same features and pipeline structure.

We configured the model with 50 trees (**numTrees=50**) and used the same preprocessing pipeline as for the GBT model. The training pipeline included: string indexing and one-hot encoding for categorical features, feature vector assembly, training a random forest on the assembled features. The model was trained on the same 80%-20% train-test split and evaluated using RMSE.

Results and Model Deployment

The performance of both models was evaluated using the Root Mean Squared Error (RMSE) on the test set:

- **Gradient Boosted Trees Regressor (GBT):** RMSE = 787.27 €/m²
- **Random Forest Regressor (RF):** RMSE = 1177.45 €/m²

The GBT model significantly outperformed the Random Forest baseline, achieving a lower prediction error. This confirms its superior ability to capture subtle and nonlinear dependencies in the data, critical for accurately modeling real estate prices in a diverse urban market like Barcelona.

To track experiments, log metadata, and facilitate reproducibility, we used **MLflow**. For each model, we recorded: Model type and key hyperparameters (e.g., **maxIter=100** for GBT, **numTrees=50** for RF), the corresponding RMSE value, the trained Spark ML pipeline artifact.

We registered both models in an MLflow model registry. The best-performing model, the **GBTRegressor**, was automatically promoted to the **Production** stage based on its evaluation score.

C: Bonus – Job Orchestration via Airflow

As an orchestration framework, we used Apache Airflow to schedule, monitor, and run our Spark-based Python scripts in an automated fashion. The DAG consists of three sequential **BashOperator** tasks according to our pipeline:

- Step 1: Format the raw data into consistent, normalized tables,
- Step 2: Enrich the formatted data and move it to the Exploitation Zone,
- Step 3: Train and evaluate models, then deploy the best-performing one using MLflow.

The DAG file is provided in `Codes/data_pipeline_dag.py` and uses Airflow's scheduling and dependency management to ensure reliable and repeatable execution. Further instructions on how to run it can be found in the GitHub README.

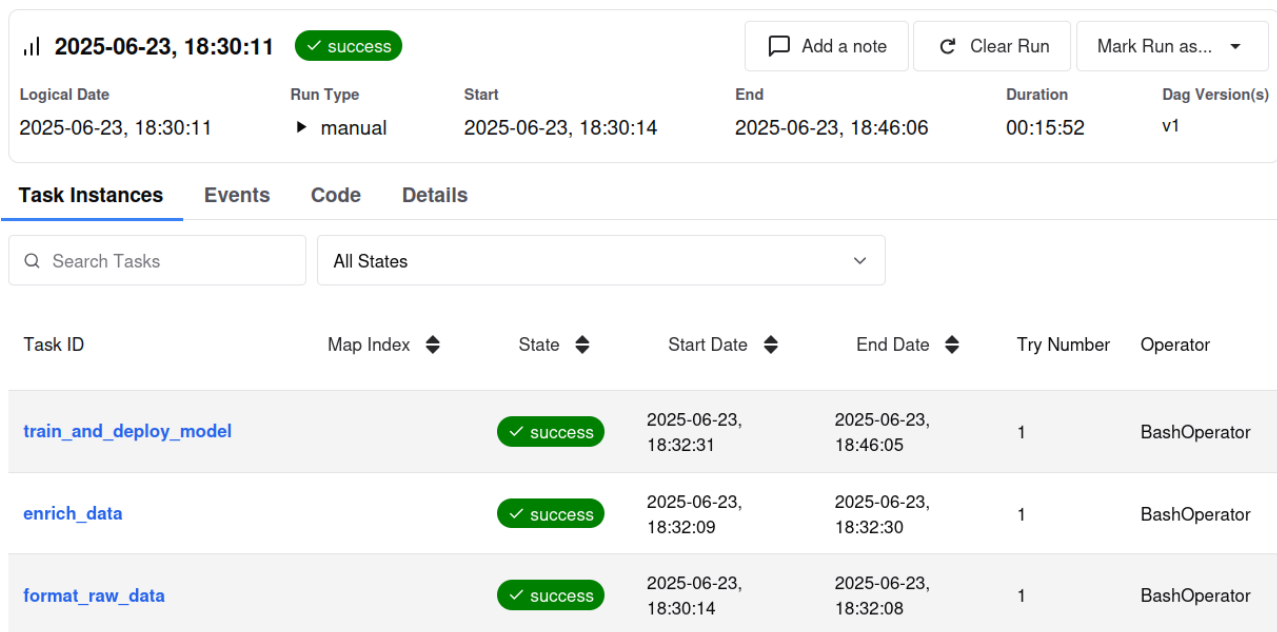


Figure 2: Airflow task instance run logs